

DSA: Assignment

Abstract

In this assignment you will learn with how to pragmatically handle graphs. Your first task is to optimally store the graphs, and find a single path between the vertices of a directed unweighted graph. You will then go one step further and find all possible paths using a different algorithm.

1 Tasks

⚠ **Be careful using global variables when working on your assignments.** In order to test your implementation `pytest` calls your functions multiple times with different parameters but in the same runtime. Make sure to initialize any global variable you use appropriately.

⚠ **Do not change function names or else `pytest` will not work.** In order to test your implementation the function names are hardcoded in the automated testing environment. **Do not change the function names in the boiler plates.** If you change the function names `pytest` will not work, you will not be able to hand-in your assignment and your submission will result in **0 pt**.

1.1 Task 1: Find One Path (3 pt)

You work for Twitter, who has made major investments to fight fake news about COVID-19 spreading through their networks. The company has given you a sample of a labelled dataset of tweet exchanges between their customers (TwitterUserID, list of users following IDs). Your task is to develop an algorithm to analyse reachability in this data set. For starters, you are tasked to find if a certain user (TwitterID-A) could potentially see a post from another user (TwitterID-B).

Allowed data structures 1. tuple, dict, list, set

Data format 1. The `find_path()` function takes three parameters: two integers representing the start and end node ids, and, the list of the twitter users with the users they are following. Each twitter user (node) is given as a tuple (`tuple[id:int, following:list[int]]`).

(i.e. id_1 follows $id_{1,1}$ and $id_{1,2}$ meaning you have the directed edges $(id_1, id_{1,1})$ and $(id_1, id_{1,2})$)

$[(<id_1>, [<id_{1,1}>, <id_{1,2}>, \dots]), \dots, (<id_n>, [<id_{n,1}>, <id_{n,2}>, \dots])]$

The function returns a list of integer `node_ids` of the path between the start and end nodes.

$[<id_{start}>, \dots, <id_{end}>]$

Call: `find_path(1, 5, [(1, [2]), (2, [3]), (3, [4]), (4, [5]), (5, [])])`

Returns (`list[int]`): `[1, 2, 3, 4, 5]`

1.2 Task 2: Find All Paths (3 pt)

Your task is to find all the possible paths from which a (TwitterID-A) could have seen a post by a user (TwitterID-B). For each path a vertex should be visited only once as the graph may contain cycles.

Allowed data structures 2. The data structures allowed are the same as for [Task 1](#).

Data format 2. The `find_paths()` function takes the same parameters and data format as [Task 1](#). But, returns a list of lists (`list[list[int]]`) of all possible paths from `start` to `end`.

$$[[id_{start}, \dots, <id_{end}>], [<id_{start}>, \dots, <id_{end}>], \dots]$$

Call: `find_paths(1, 3, [(1, [2, 3]), (2, [3, 1]), (3, [2])])`

Returns (`list[list[int]]`): `[[1, 3], [1, 2, 3]]`

1.3 Task 3: Shortest Path (4 pt)

Develop an algorithm that finds the shortest path between a user (TwitterID-A) spreading fake news and some other user (TwitterID-B).

Allowed data structures 3. The data structures allowed are the same as for [Task 1](#).

Data format 3. The `shortest_path` function takes three parameters: two integers representing the start and end node ids, and, the list of the twitter users with their number of likes and the users they are following. Each twitter user (node) is given as a tuple (`tuple[id:int, num_likes:int, following:list[int]]`).

All edges directed to a node u have a weight of $1/\text{num_likes}_u$

$$[(<id_1>, <\text{num_likes}_1>, [<id_{1,1}>, <id_{1,2}>, \dots]), \dots, (<id_n>, <\text{num_likes}_n>, [<id_{n,1}>, <id_{n,2}>, \dots])]$$

The function returns a list of integer `node_ids` of the shortest paths between the start and end nodes.

$$[<id_{start}, \dots, <id_{end}>]$$

Call: `shortest_path(1, 3, [(1, 2, [2, 3]), (2, 5, [3, 1]), (3, 6, [2])])`

Returns (`list[int]`): `[1, 3]`